

Cross-AI, Lean-Verified Mathematics: A Case Study on the Collatz Conjecture (v5 — methodology retrospective and honest close)

Piero Borgatta*

June 5, 2026

Abstract

This is the fifth and closing version of a personal research program on the Collatz conjecture. Its primary subject is the program’s *explicitly declared secondary goal* from v2 onward: testing how far iterative, cross-checked collaboration with large language model (LLM) assistants can carry a non-standard attempt on an open mathematical problem. The mathematics is reported honestly as a closed (but not abandoned) chapter, and the methodology is brought to the foreground.

We report, with reproducible figures, what a multi-AI workflow (Google Gemini, Anthropic Claude Code, OpenAI Codex, and an external DeepSeek check through `aider-desk`) produced under the author’s direction over a five-week span: a mechanically verified, `sorry-/admit-/axiom-free` Lean 4 + Mathlib core (an exact congruential shadowing lemma, a no-infinite-shadowing corollary, an expanding-cycle exclusion, finite Collatz–Wielandt spectral-radius certificates, and an elementary descent bridge); a finite phantom-set taxonomy; and roughly 1.4×10^5 lines of AI-produced code and prose, of which the author typed none by hand. We also report the program’s *negative* results honestly — the analytic routes that hit documented barriers, including a cross-AI episode in which the assistants generated, and then demolished, four “revolutionary” proposals — and we record the methodological observation we consider most interesting: a cross-AI loop, when one side is anchored to concrete obstructions, can *recognize and prove the ceiling of its own approach*. The closest external comparison is Chang’s 2026 human–LLM Collatz program [2], which is broader, not Lean-centered, and independently locates the same distributional-to-pointwise obstruction; we use it here as a methodological comparator rather than as a priority foil. We distill transferable lessons and failure modes for AI-assisted theorem proving. The mathematical frontier the program identified as the only productive one (cycle exclusion) is left explicitly open as future work, not closed. We do not claim a proof of the Collatz conjecture.

Contents

1	The v1→v5 arc	2
2	The verified artifact	2
3	Quantitative summary	3
4	Methodology: the cross-AI protocol	4
5	External comparator: Chang’s human–LLM Collatz program	5

*Independent researcher. info@pieroborgatta.com; ORCID 0009-0001-8025-2405.

6	What worked	6
7	What failed — honest failure modes	7
8	The limits, recognized and proved	7
9	Open mathematical direction (left open, not closed)	8
10	Lessons for AI-assisted mathematics	8
11	Reproducibility	8
12	Conclusion	9

1 The v1→v5 arc

This program began (v1) as an ambitious proposal: a spectral reduction of the Collatz conjecture via “phantom orbit shadowing,” framed as a *possible route to a resolution*. It then evolved honestly:

- **v1** — the original spectral program, proposed as a possible Collatz resolution.
- **v2** — the shadowing core mechanically verified in Lean 4; comparison with the concurrent work of [2]; the AI-collaboration methodology declared as an explicit secondary goal.
- **v3** — a conditional spectral reduction, a phantom-set taxonomy at depth $K_0 = 16$, and an extended Lean operator/certificate layer.
- **v4** — a consolidation that *proved* the program’s two analytic branches cannot reach the conjecture, mapping the two distinct barriers and isolating the open content precisely [1].
- **v5** (this document) — the secondary goal becomes the subject: an honest methodology retrospective and the program’s close, with the verified artifacts as evidence and the productive mathematical frontier left open.

We regard this arc — from “possible solution” to a precise, honest map of *why* the approach stops and *what* is nonetheless verified — as the scientifically valuable shape of the work, more so than any single intermediate claim. Reporting it candidly, including the failures, is the point of v5.

2 The verified artifact

All of the following is formalized in Lean 4 + Mathlib and is **sorry**-, **admit**- and **axiom**-free (verified by `grep` over the `CollatzShadowing` sources and by `lake build`).

- `exact_shadowing`, `exact_shadowing_periods` — the exact congruential shadowing lemma.
- `no_infinite_period_congruence_expansive` — no infinite periodic shadowing.
- `no_positive_endpoint_eventually_periodic_expansive_congruence` — no positive integer on an expanding cycle (post-prefix form).

- `spectralRadius_le_of_finiteCWCertificate` — the generic weighted Collatz–Wielandt bound, bridged to Mathlib’s real `spectralRadius`.
- `t10j32HighBitTailSpectralRadiusBound_97_2000` (single node, $\leq 97/2000$) and `k16s16KDeterministicGeneratedSpectralRadiusBound` (deterministic (K, b) bound $< 3/4$ at $K_0 = 16$).
- `classicalCollatz_of_uniformStrictDescent_provedBridge` — the elementary descent bridge to the classical conjecture.

The full declaration map is in `lean/CollatzShadowing/THEOREM_INDEX.md`; the build is reproducible from `lean/` via `lake exe cache get` then `lake build`.

3 Quantitative summary

All figures are read-only measurements of the repository (`git`, `wc`, `grep`); reproduction commands and the full breakdown are in `notes/PROJECT_STATISTICS.md`.

Authorship. Every file in the repository — all code and all prose — was produced by the AI assistants under the author’s direction. **The author typed none of it by hand**; his contribution is direction, conceptual framing, decisions, editing-by-instruction, and all external actions (publication, compilation, version control operations). This should be read operationally: the repository is an AI-authored artifact under human experimental control, not an autonomous AI theorem.

category	lines	content
AI-authored source	$\approx 61,800$	Lean proofs 7,805 + Python 54,009
generator-emitted	43,513	Lean finite certificates (script output)
prose	35,673	working notes 28,907 + LaTeX 6,766
total (AI-produced)	$\approx 141,000$	human-typed: 0

- **Lean:** 40 files; 302 AI-authored theorems/lemmas (14 modules) plus 966 emitted by generators (26 certificate modules); *zero sorry/admit/axiom*.
- **Python:** 151 scripts (all tracked, none missed) — spectral program 83, phase-one empirical 42, phantom taxonomy 16, root-level 10.
- **History:** 84 commits over 37 days (≈ 5.3 weeks); commit authorship (via `.mailmap`) Codex 52, Piero Borgatta 35.
- **Honesty caveat:** never quote the 51k Lean total or the 1,268 theorem count without the AI-authored vs generator-emitted split; the generator output dominates both.

Soft metrics. Human hours, number of AI sessions, and message volume are not cleanly measurable from the repository and are reported separately as labelled estimates; we deliberately do *not* state an “AI work hours” figure, since LLM inference time is not comparable to human labour hours. This missing instrumentation is a limitation of the case study. Future work should log, from the beginning, model identity, wall-clock session duration, turn counts, approximate token use, task category, verification result, and whether a model’s output survived cross-AI or Lean review.

Human control points. The “zero typed lines” claim is therefore not a claim of zero intellectual contribution. It is a claim about the physical production channel of the artifact. The human contribution is visible in a different set of control variables: choosing the research target, deciding when to publish or retract claims, requiring `lake build` and a `sorry`-free state, asking for adversarial checks rather than agreement, redirecting the program after negative evidence, selecting the final framing, and performing all external actions. A reviewer may reasonably ask what the author contributed if no line was typed by hand; the answer is that the author controlled the experiment, the stop/go decisions, and the scientific claims, while the assistants produced the textual and code-level substrate.

4 Methodology: the cross-AI protocol

The working configuration was:

1. **Human direction.** The author set the research direction and framing, made all strategic and publication decisions, and performed all external actions.
2. **Generate / critique split.** The assistants alternated roles: one proposing constructions or proofs, the other auditing them against the concrete state of the problem.
3. **Lean as ground truth.** Every formal claim was funnelled through Lean: a proof counts only if `lake build` succeeds with no `sorry`. This converts “plausible-looking” AI output into a binary, machine-checked verdict — the single most important discipline of the project.

Operational toolchain. The actual workflow was not a symmetric two-model contest. It was an orchestrated toolchain with different roles:

tool	role in this project
Google Gemini	Early high-level, deliberately free-form speculative idea generation, always under the author’s direction and starting from the author’s conceptual prompts.
Claude Code (Opus 4.7/4.8)	Validation, refinement, pruning of superficial Gemini ideas, later proof planning, manuscript drafting, and comparison against external critiques.
OpenAI Codex	Repository-operational agent: continuation of selected ideas, implementation of Python scripts, Lean code, generated certificates, builds, TODO files, Markdown notes, patching, local checks, and compilation support.
<code>aider-desk</code> + DeepSeek v4 Flash API	External-opinion channel. Its critique was not treated as authoritative; it was fed back to Claude Code for adversarial comparison before returning to Codex for implementation or rejection.
Overleaf / Zenodo / GitHub	External publication layer. Once the author judged that a paper version was worth writing, Claude Code often drafted the $\text{\LaTeX}$, Overleaf was used for online compilation in some versions, and the author or an AI coding agent performed pushes. The author made all publication decisions and uploaded to Zenodo.

The resulting loop was roughly:

human idea/direction → Gemini exploration → Claude critique → Codex implementation
→ DeepSeek external opinion → Claude comparison → Codex revision/build.

Not every cycle used every tool, but this was the characteristic pattern.

Why Lean mattered. For the author, Lean was not decorative. It was the only reliable way to know whether a plausible AI-produced mathematical statement had survived contact with a formal checker. Cross-AI criticism improved the search process, but it did not certify mathematics. The certification boundary in this project is therefore explicit: Lean-backed claims are machine-checked; the remaining analytic diagnoses and methodological conclusions are paper-level arguments, not formalized theorems.

Subscription and cost disclosure. During the project the author had active, general-purpose AI subscriptions or credits: Claude at roughly EUR 20/month; Gemini at roughly EUR 20/month; ChatGPT/OpenAI at roughly EUR 20/month initially and later roughly EUR 100/month as usage increased; and about EUR 50 of DeepSeek API credit through the `aider-desk` channel. These figures are *not* the cost of this research project. The same subscriptions were used simultaneously for ordinary programming, Python scripts, websites, iOS and Android/React app work, image generation for the author’s employer, generic tasks, VBA scripts, and other unrelated work. We therefore report them only as environment disclosure, not as an attributable project budget.

The decisive methodological lesson (Section 8) is that the generate/critique split is productive *only when the critic is anchored to concrete objects and obstructions*, not to labels. The same kind of model, in the generative role, repeatedly produced plausible but structurally empty proposals; in the critic role, with the problem’s barriers loaded as context, it dismantled them. The asymmetry is one of *role and discipline*, not of model capability.

5 External comparator: Chang’s human–LLM Collatz program

The nearest external comparator is Edward Y. Chang’s *Exploring Collatz Dynamics with Human–LLM Collaboration* [2]. The current arXiv v6 (updated 22 April 2026) is explicitly not a proof of the Collatz conjecture. It is instead a much broader human–LLM structural survey: a 233-page account reporting 630 formal results, roughly 10^{14} computational experiments, and 29 distinct paradigms. Its methodology section describes a three-party workflow in which Claude acted primarily as a theorem/construction generator, GPT as a validator and local framework checker, and the human as moderator and strategic relay.

dimension	Chang v6	this project v5
scope	Broad structural survey: 29 paradigms, many aggregate routes	Narrow verified case study around shadowing, finite certificates, and the first-barrier stop rule
verification surface	Paper-level proofs and exact computational evidence; not presented as a Lean/Coq/Isabelle formalization	Lean 4 + Mathlib core with <code>lake build</code> and no <code>sorry/admit/axiom</code>
main obstruction	Distributional-to-pointwise gap for promoting aggregate control to individual orbits	Same gap localized in the phantom/2-adic objects via the non-transport/rigidity analysis
methodology record	Rich transcript/attack-plan/error-correction accounting	Reproducible repository metrics plus binary proof-assistant verification
best read as	A map of many AI-generated Collatz attack surfaces	A narrow machine-checked artifact plus a negative stopping rule

Mathematical overlap. The overlap is real and should be stated plainly. Both programs use the accelerated Syracuse map, 2-adic coordinates, valuation words, and phantom cycles. Chang’s phantom root attached to a finite parity block is the same mathematical object as this project’s rational $q_w = -C_w/(3^L - 2^A)$, viewed through the 2-adic fixed-point equation for the block map. Chang’s repulsion statements for finite integer orbits near phantom roots are mathematically adjacent to the Lean-checked exact shadowing and no-infinite-shadowing results reported here, though the formal surfaces are different.

The more important agreement is negative. Chang isolates a distributional-to-pointwise gap: aggregate, density-one, cylinder-averaged, or transfer-operator contraction information does not automatically produce a theorem for each individual integer. v4 of this project reaches the same barrier inside its own objects: the aperiodic rigidity needed to turn the phantom/2-adic picture into a pointwise no-divergence theorem is essentially equivalent to the divergence half of Collatz itself (conditional on the standard Lagarias coding). In both projects, the obstruction is not a failed constant, a too-small computation, or a missing clever norm; it is a logical gap between averaged control and pointwise orbit control.

Different output geometry. Chang’s project is broad by design. It maps many paradigms: spectral, martingale, discrete-log, formal-language, cascade, density, cylinder, and cycle-exclusion analyses. Its strength is coverage and error-correction documentation. It is the better source for seeing how many attractive AI generated routes to Collatz collapse into known structural barriers.

This project is narrower and deeper at the level of machine verification. It does not survey 29 paradigms. Instead, it turns one small core into a reproducible Lean 4 artifact: exact shadowing, no infinite periodic shadowing, finite Collatz–Wielandt certificates, an expanding-cycle exclusion, and the descent bridge. The defensible novelty claim is therefore not “first AI-assisted Collatz study”; Chang makes that framing untenable. The defensible claim is more precise: a case study in which a cross-AI loop produced a `sorry`-free Lean core and then localized the limits of the approach with enough precision to stop making stronger claims.

Methodological comparison. Chang is also useful because it records collaboration data that this project only partially records: attack-plan histories, transcript-based error-correction logs, and even approximate division-of-labour profiles for some windows. This paper should learn from that practice. The repository statistics here are reproducible, but the session-level variables remain soft metrics. We therefore report them only as labelled estimates and separate them sharply from the machine-checkable build facts.

Conversely, this project’s methodological discipline is stronger where a formal theorem is claimed: Lean is the arbiter. Cross-AI agreement is not treated as evidence. A statement enters the verified core only when `lake build` succeeds without `sorry`, `admit`, or `axiom`. The comparison with Chang therefore sharpens, rather than weakens, the message of v5: broad human–LLM exploration is valuable for mapping attack surfaces, while proof assistants are essential when the output is to be counted as mathematics rather than plausible research conversation.

6 What worked

- Producing and *mechanically verifying* a real (if elementary) body of mathematics: the shadowing core and the finite certificates.

- Managing a large, multi-week formalization with a coherent module structure and a clean `sorry`-free state.
- Generating exact finite certificates programmatically and importing them into Lean with machine-checked row witnesses.
- Cross-AI auditing that repeatedly caught Mathlib API mismatches and Lean 3-vs-4 syntax errors one assistant alone missed.

7 What failed — honest failure modes

This is, in our view, the most useful part of the report: most accounts of “LLMs doing mathematics” present only successes.

- **Abandoned analytic branches.** A $\mathbb{Z}_2$ martingale Banach space (failed: negative 2-adic child-cylinder signal); an A0 weak averaged bridge and an A1 refined-square kernel (partial; barrier); a pointwise first-barrier branch (conditional; negatively resolved in v4).
- **“Prestige-transfer” proposals.** In a documented cross-AI episode, the assistants generated four “revolutionary” pivots (Galois/anabelian action on the inverse tree; non-standard models / independence; “exotic” Banach spaces; holomorphic extension), each of which was then dismantled with a specific structural reason. The common failure: each imports rigidity from a richer ambient category (algebraic, logical, measure-analytic, complex-analytic), but the hard invariant — an individual integer carrying an aperiodic 2-adic valuation word — is precisely what every such embedding renders generic or invisible.
- **Label pattern-matching.** Concrete instances: proposing Mathlib contributions based on *removed* API (`Nat.bit0/bit1`); hallucinated lemma names; assuming novel general-purpose lemmas where the project had in fact *assembled* existing Mathlib primitives.
- **Generative overreach.** A persistent tendency to connect semantically adjacent domains without checking structural compatibility against the actual objects.

8 The limits, recognized and proved

The program’s strongest methodological outcome is that the cross-AI loop did not merely fail quietly; it *localized and proved its own ceiling*.

- **Two barriers, separated.** v4 establishes that Collatz splits into two distinct hard halves: *no divergent orbits* (barrier: distributional-to-pointwise) and *no nontrivial cycles* (barrier: Diophantine approximation of $\log_2 3$). The program’s entire analytic effort lived in the first.
- **A negative structural result.** Within the phantom/2-adic formalism, the rigidity one would need on the orbit limit point is *logically equivalent* to the divergence conjecture itself (a non-transport observation: the parity-vector conjugacy carries eventual periodicity to dynamical pre-periodicity, not to rationality; conditional on the standard Lagarias coding).

That an AI-assisted process can arrive at, and rigorously articulate, “this approach cannot work, and here is why” — rather than continuing to produce hopeful variations — is a result we did not anticipate and consider the central methodological finding.

9 Open mathematical direction (left open, not closed)

The program is closed as a methodology study, but the mathematics it pointed to is *not abandoned*. v4 identified one genuinely productive frontier, and we keep it explicitly open as future work:

The **cycle half** (no nontrivial cycles). This is the only half whose tools (linear forms in logarithms; continued fractions of $\log_2 3$; computational verification; [3, 4, 5, 6]) genuinely bite. The realistic, finishable target is the *structural layer* formalized in Lean — the cycle equation $n(2^A - 3^L) = C_w$, the contractivity condition $2^A > 3^L$ for positive cycles, the Diophantine condition on A/L , and the link to computational verification — with the deep analytic exclusions left as named hypotheses, since a usable form of Baker’s theorem is not currently in Mathlib.

Anyone (the author included) may resume the program from this point. The verified Lean library (Section 2) is the natural foundation for that continuation.

10 Lessons for AI-assisted mathematics

1. **Mechanical verification is the load-bearing discipline.** Lean’s binary verdict is what separates this program’s verified core from the unverifiable analytic speculation; AI prose proofs without it are not evidence.
2. **Cross-AI value is adversarial, not additive.** The benefit came from one model auditing another against concrete obstructions, not from two models agreeing.
3. **Generative fluency is not mathematical judgment.** The same system produces both verified proofs and structurally empty “revolutionary” proposals; the discriminator is anchoring to objects, not to vocabulary.
4. **Honest negative results are a deliverable.** Localizing and proving an approach’s ceiling saved further wasted effort. The deliverable is not merely “we failed”; it is a reproducible map of which hypotheses would be needed, why the present machinery cannot supply them, and where any continuation must move.
5. **Beware inflated metrics.** Report AI-authored vs generated separately; do not fabricate “AI hours.”

In this project, the negative result mattered more than another finite certificate would have mattered. It converted an attractive research narrative into a bounded one: the first-barrier branch can still be studied as operator approximation, but it cannot honestly be sold as a direct Collatz route without a new pointwise idea. For AI-assisted mathematics, that kind of stopping rule is a positive capability.

11 Reproducibility

The Lean 4 library, the Python scripts, the working notes (including PROJECT_STATISTICS.md with reproduction commands), and the paper sources are in the supplementary archive and at <https://github.com/PieroBorgatta/Collatz>. The verified Lean library is additionally published as a standalone Lake package at <https://github.com/PieroBorgatta/CollatzShadowing> (release v0.1.0), with the

`lakefile` at the repository root for indexing in the Lean package registry (Reservoir). This version (v5): [10.5281/zenodo.20554750](https://doi.org/10.5281/zenodo.20554750). Earlier versions: concept DOI [10.5281/zenodo.20021537](https://doi.org/10.5281/zenodo.20021537); v4 [10.5281/zenodo.20544464](https://doi.org/10.5281/zenodo.20544464).

12 Conclusion

Over five weeks, a multi-AI workflow under human direction produced a mechanically verified mathematical core, a precise map of two barriers blocking the Collatz conjecture, an honest catalogue of failed routes, and — the result we find most interesting — a proof-level diagnosis of its own approach’s ceiling (Lean-checked for the formal core, paper-level for the non-transport/gap analysis). We do not claim progress on the conjecture. We claim a verified artifact, a candid account of what cross-AI mathematical collaboration can and cannot do, and a clearly delimited open frontier for whoever continues.

References

- [1] P. Borgatta, *Phantom Orbit Shadowing for the Collatz Conjecture: Verified Core, Two Barriers, and a Repositioning* (v4), Zenodo, DOI [10.5281/zenodo.20544464](https://doi.org/10.5281/zenodo.20544464), 2026.
- [2] E. Y. Chang, *Exploring Collatz Dynamics with Human-LLM Collaboration*, arXiv:2603.11066v6, 2026.
- [3] R. P. Steiner, *A theorem on the Syracuse problem*, Proc. 7th Manitoba Conf. on Numerical Mathematics, 1977, 553–559.
- [4] J. L. Simons, *On the nonexistence of 2-cycles for the $3x + 1$ problem*, Math. Comp. **74** (2005), 1565–1572.
- [5] J. L. Simons and B. M. M. de Weger, *Theoretical and computational bounds for m -cycles of the $3n + 1$ problem*, Acta Arith. **117** (2005), 51–70.
- [6] D. Barina, *Convergence verification of the Collatz problem*, J. Supercomput. **77** (2021), 2681–2688.